Analyse en composantes principales et décomposition en valeurs singulières

Yassine Abderrezak Dache
Projet personnel – Python (NumPy, Matplotlib)
https://github.com/yassinedache/pca-svd-from-scratch

6 octobre 2026

Résumé

On programme deux algorithmes de rotations de Jacobi, l'un pour diagonaliser une matrice symétrique et l'autre pour calculer la décomposition en valeurs singulières (SVD), sans faire appel aux routines de NumPy. Ils servent d'abord à réaliser l'analyse en composantes principales (ACP) du jeu de données *Wine* par deux voies, la diagonalisation de la matrice de covariance puis la SVD de la matrice des données : les deux voies coïncident à $3{,}1 \times 10^{-15}$ près, et le premier plan factoriel sépare les trois cépages (97,2 % de vins bien classés par une règle du centre le plus proche). On montre ensuite pourquoi la voie SVD est numériquement préférable. Enfin, on compresse une image 512×512 par approximation de rang faible et l'on vérifie le théorème d'Eckart–Young : l'erreur mesurée coïncide avec la formule théorique à $3{,}5 \times 10^{-15}$ près.

1 Rappels : diagonalisation et SVD

Théorème 1 (Théorème spectral)**.** *Toute matrice symétrique réelle $S \in \mathbb{R}^{p\times p}$ s'écrit $S = V\Lambda V^\top$ avec V orthogonale et $\Lambda = \mathrm{diag}(\lambda_1 \geq \cdots \geq \lambda_p)$.*

Théorème 2 (Décomposition en valeurs singulières)**.** *Toute matrice $A \in \mathbb{R}^{m\times n}$ s'écrit $A = U\Sigma V^\top = \sum_{i=1}^{r} \sigma_i\, u_i v_i^\top$ avec U, V à colonnes orthonormées, $\sigma_1 \geq \cdots \geq \sigma_r > 0$ et $r = \mathrm{rg}\, A$.*

Les deux décompositions sont liées : $A^\top A = V\Sigma^2 V^\top$. Les valeurs singulières de A sont donc les racines carrées des valeurs propres de $A^\top A$, et ses vecteurs singuliers à droite sont les vecteurs propres de $A^\top A$.

1.1 Méthode de Jacobi pour une matrice symétrique

Une rotation de Givens $J(p, q, \theta)$ agit seulement sur les lignes et colonnes p et q. On choisit θ pour annuler le coefficient s_{pq} de $J^\top S J$:

$$\tan 2\theta = \frac{2s_{pq}}{s_{qq} - s_{pp}}.$$

Comme la norme de Frobenius est invariante par rotation, la masse hors diagonale $\mathrm{off}(S)^2 = \sum_{i\neq j} s_{ij}^2$ diminue exactement de $2s_{pq}^2$ à chaque rotation. En balayant tous les couples (p, q), $\mathrm{off}(S) \to 0$ (la convergence est même quadratique en fin de calcul), et le produit des rotations donne les vecteurs propres.

1.2 SVD par la méthode de Jacobi « à un côté »

Au lieu de diagonaliser $A^\top A$, on applique des rotations à droite de A : pour un couple de colonnes (a_p, a_q), on choisit la rotation qui les rend orthogonales. Cela revient à appliquer la méthode précédente à $A^\top A$, mais sans jamais former ce produit. Quand toutes les colonnes de AV sont orthogonales, on a $AV = U\Sigma$: les normes des colonnes sont les valeurs singulières. Pour rendre l'algorithme rapide en Python, on traite à chaque étape $n/2$ couples disjoints simultanément (ordre « round-robin ») avec des opérations vectorisées de NumPy. La SVD de l'image 512×512 prend ainsi 17,2 s, et ses valeurs singulières diffèrent de celles de NumPy de $7{,}6 \times 10^{-14}$ en relatif.

2 Analyse en composantes principales

2.1 Deux façons de la calculer

Soit $X \in \mathbb{R}^{n\times p}$ la matrice des données centrées (et réduites), une ligne par individu. L'ACP cherche les directions v de variance maximale : la variance des projections Xv vaut $v^\top C v$, avec $C = \frac{1}{n-1} X^\top X$ la matrice de covariance (ou de corrélation pour des données réduites).

Proposition 1. *Le maximum de $v^\top C v$ sous la contrainte $\|v\| = 1$ vaut λ_1 et il est atteint pour le premier vecteur propre v_1 de C. Plus généralement, les axes principaux sont les vecteurs propres $v_1, \ldots, v_p$ et la variance le long de v_j est λ_j.*

Démonstration. On décompose $v = \sum_j c_j v_j$ dans la base orthonormée des vecteurs propres. Alors $v^\top C v = \sum_j \lambda_j c_j^2 \leq \lambda_1 \sum_j c_j^2 = \lambda_1$, avec égalité pour $v = v_1$. On obtient les axes suivants en répétant le raisonnement dans l'orthogonal des axes déjà trouvés. □

1. **Voie covariance** : on forme C puis on la diagonalise par la méthode de Jacobi.
2. **Voie SVD** : on calcule $X = U\Sigma V^\top$. Comme $C = V \frac{\Sigma^2}{n-1} V^\top$, les axes principaux sont les colonnes de V, les variances sont $\lambda_j = \sigma_j^2/(n-1)$, et les coordonnées des individus sont $XV = U\Sigma$.

2.2 Application au jeu de données Wine

Le jeu de données *Wine* (UCI, Aeberhard et Forina) décrit 178 vins italiens issus de trois cépages par 13 mesures chimiques (alcool, acide malique, flavanoïdes, intensité de la couleur, proline, etc.). Les variables ont des unités et des ordres de grandeur très différents, on les réduit donc avant l'analyse. Sans cette réduction, la première composante expliquerait 99,8 % de la variance et ne serait presque que la variable `proline`, dont les valeurs sont de l'ordre de 1 000.

Axe	λ_j (covariance, Jacobi)	$\sigma_j^2/(n-1)$ (SVD)	Variance expliquée	Cumul
CP_1	4,7059	4,7059	36,2 %	36,2 %
CP_2	2,4970	2,4970	19,2 %	55,4 %
CP_3	1,4461	1,4461	11,1 %	66,5 %
CP_4	0,9190	0,9190	7,1 %	73,6 %
CP_5	0,8532	0,8532	6,6 %	80,2 %
CP_6	0,6417	0,6417	4,9 %	85,1 %

Table 1 – Les deux voies donnent les mêmes valeurs à $3{,}1 \times 10^{-15}$ près (et la même chose que `scikit-learn`, à $1{,}2 \times 10^{-14}$ près). Pour des données réduites, $\sum_j \lambda_j = \operatorname{tr} C = 13$.

Les deux premières composantes résument 55,4 % de l'information et les cinq premières 80,2 %. Le critère de Kaiser (conserver les axes de valeur propre supérieure à 1, soit plus d'information qu'une variable initiale) retient 3 composantes.

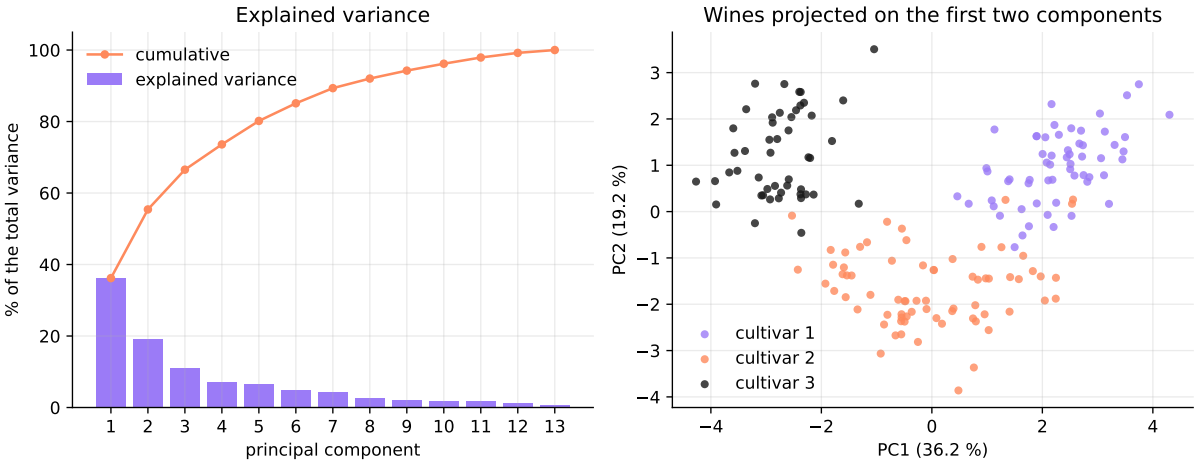
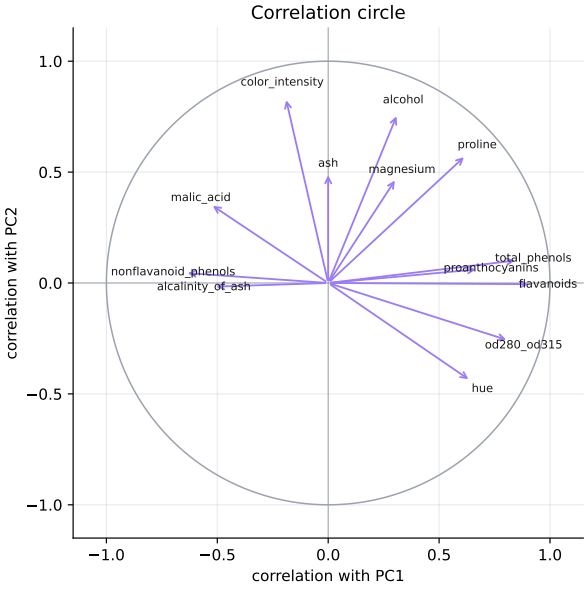


Figure 1 – Éboulis des valeurs propres et projection des 178 vins sur le premier plan factoriel.

2.3 Interprétation

Pour des données réduites, la corrélation entre la variable j et la composante k vaut $v_{jk}\sqrt{\lambda_k}$. Le cercle des corrélations permet de nommer les axes.

Variable	CP_1	CP_2
`flavanoids`	+0,92	−0,01
`total_phenols`	+0,86	+0,10
`od280_od315`	+0,82	−0,26
`proanthocyanins`	+0,68	+0,06
`nonflavanoid_phenols`	−0,65	+0,05
`hue`	+0,64	−0,44
`proline`	+0,62	+0,58
`malic_acid`	−0,53	+0,36
`alcalinity_of_ash`	−0,52	−0,02
`alcohol`	+0,31	+0,76
`magnesium`	+0,31	+0,47
`color_intensity`	−0,19	+0,84
`ash`	+0,00	+0,50



La première composante oppose les vins riches en composés phénoliques (flavanoïdes, phénols totaux, rapport OD280/OD315) à ceux qui ont davantage de phénols non flavanoïdes et d'acide malique. La deuxième est portée par l'intensité de la couleur, le degré d'alcool et la proline. Dans ce plan, les trois cépages forment trois groupes distincts : le cépage 1 a des scores élevés sur les deux axes, le cépage 3 un score faible sur le premier, et le cépage 2 se situe en bas. Une règle simple, qui affecte chaque vin au cépage dont le centre est le plus proche dans ce plan (validée en laissant chaque vin de côté tour à tour), classe correctement 97,2 % des vins, alors que l'ACP n'a jamais utilisé l'étiquette des cépages.

2.4 Pourquoi préférer la voie SVD

Former $X^\top X$ élève le conditionnement au carré : une valeur singulière σ devient la valeur propre σ^2, et si σ^2 est inférieure à $\varepsilon\,\sigma_1^2$ (avec $\varepsilon \approx 10^{-16}$ la précision machine), elle est noyée dans

les erreurs d'arrondi. On le vérifie sur la matrice de Läuchli

$$A = \begin{pmatrix} 1 & 1 & 1 \\ \epsilon & 0 & 0 \\ 0 & \epsilon & 0 \\ 0 & 0 & \epsilon \end{pmatrix}, \qquad A^{\top}A = \begin{pmatrix} 1+\epsilon^2 & 1 & 1 \\ 1 & 1+\epsilon^2 & 1 \\ 1 & 1 & 1+\epsilon^2 \end{pmatrix},$$

dont la plus petite valeur singulière vaut exactement ϵ. Dès que $\epsilon^2 < \varepsilon$, le nombre $1+\epsilon^2$ est arrondi à 1 et l'information est perdue.

ϵ	Erreur relative (valeurs propres de $A^{\top}A$)	Erreur relative (SVD de A)
10^{-2}	$1{,}2 \times 10^{-13}$	$1{,}7 \times 10^{-16}$
10^{-4}	$5{,}7 \times 10^{-9}$	0
10^{-6}	$9{,}0 \times 10^{-7}$	0
10^{-8}	$1{,}0 \times 10^{0}$	0
10^{-10}	$1{,}0 \times 10^{0}$	0

Table 2 – La voie covariance perd la moitié des chiffres significatifs puis tous ; la SVD reste exacte.

3 Compression d'images par approximation de rang faible

3.1 Le théorème d'Eckart–Young

Une image en niveaux de gris est une matrice $A \in \mathbb{R}^{m \times n}$. On l'approche par la somme tronquée $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^{\top}$, dont le stockage demande $k(m+n+1)$ nombres au lieu de mn.

Théorème 3 (Eckart–Young–Mirsky)**.** *Pour toute matrice B de rang au plus k,*

$$\|A - B\|_F \;\geq\; \|A - A_k\|_F = \Big(\sum_{i>k} \sigma_i^2\Big)^{1/2} \quad \text{et} \quad \|A - B\|_2 \;\geq\; \|A - A_k\|_2 = \sigma_{k+1}.$$

Idée de la preuve (norme spectrale). Si $\operatorname{rg} B \leq k$, le noyau de B est de dimension au moins $n-k$; il rencontre donc le sous-espace de dimension $k+1$ engendré par $v_1, \dots, v_{k+1}$. On y choisit un vecteur unitaire $x = \sum_{i \leq k+1} c_i v_i$: alors $\|(A-B)x\| = \|Ax\| = \big(\sum_{i \leq k+1} \sigma_i^2 c_i^2\big)^{1/2} \geq \sigma_{k+1}$. Enfin, $A - A_k = \sum_{i>k} \sigma_i u_i v_i^{\top}$ a pour plus grande valeur singulière σ_{k+1}, et pour norme de Frobenius $\big(\sum_{i>k} \sigma_i^2\big)^{1/2}$ par orthonormalité. □

3.2 Expérience

On utilise une photographie 512×512 du domaine public (NASA), convertie en niveaux de gris. Le tableau compare l'erreur relative mesurée $\|A - A_k\|_F / \|A\|_F$ à la formule du théorème, calculée à partir des seules valeurs singulières. Le PSNR (rapport signal sur bruit de crête) est la mesure de qualité usuelle en traitement d'images : au-delà d'environ 30 dB, la différence devient difficile à voir.

Avec $k = 50$, on ne stocke que 19,6 % des nombres de l'image pour une erreur relative de 7,9 % et un PSNR de 27,6 dB.

Remarque. Cette compression reste théorique : les formats comme JPEG font beaucoup mieux, car ils utilisent une base fixe (cosinus) adaptée à l'œil et un codage entropique. L'intérêt de l'expérience est de rendre visible le théorème d'Eckart–Young, qui est aussi le fondement de l'ACP : les k premières composantes principales donnent la meilleure approximation de rang k du nuage de points centré.

Rang k	Données stockées	Erreur mesurée	Formule d'Eckart–Young	PSNR (dB)
1	0,4 %	0,4595	0,4595	12,1
5	2,0 %	0,2855	0,2855	16,3
10	3,9 %	0,2053	0,2053	19,2
20	7,8 %	0,1438	0,1438	22,3
50	19,6 %	0,0787	0,0787	27,6
100	39,1 %	0,0415	0,0415	33,1
200	78,2 %	0,0155	0,0155	41,7

Table 3 – L'écart maximal entre erreur mesurée et erreur théorique est de $3{,}5 \times 10^{-15}$. On a aussi vérifié que $\|A - A_k\|_2 = \sigma_{k+1}$ pour chaque k.

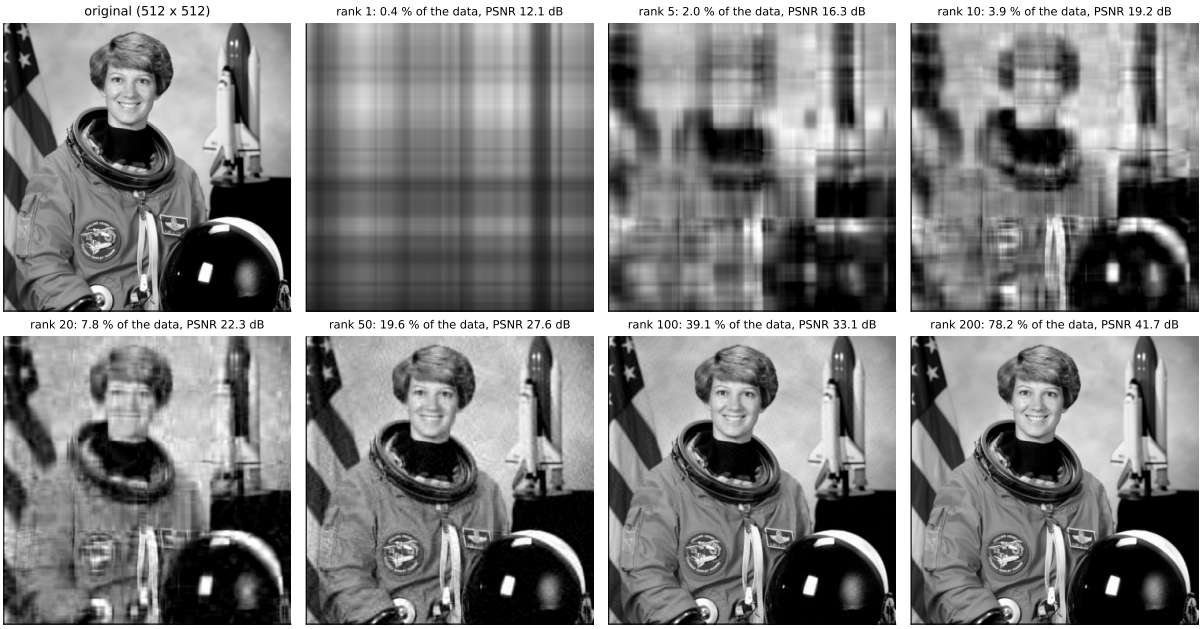


Figure 2 – Reconstructions de rang croissant. Les premières composantes ne contiennent que la structure grossière de l'image (des motifs « en croix », car chaque terme $u_i v_i^\top$ est un produit d'une colonne par une ligne) ; les détails apparaissent progressivement.

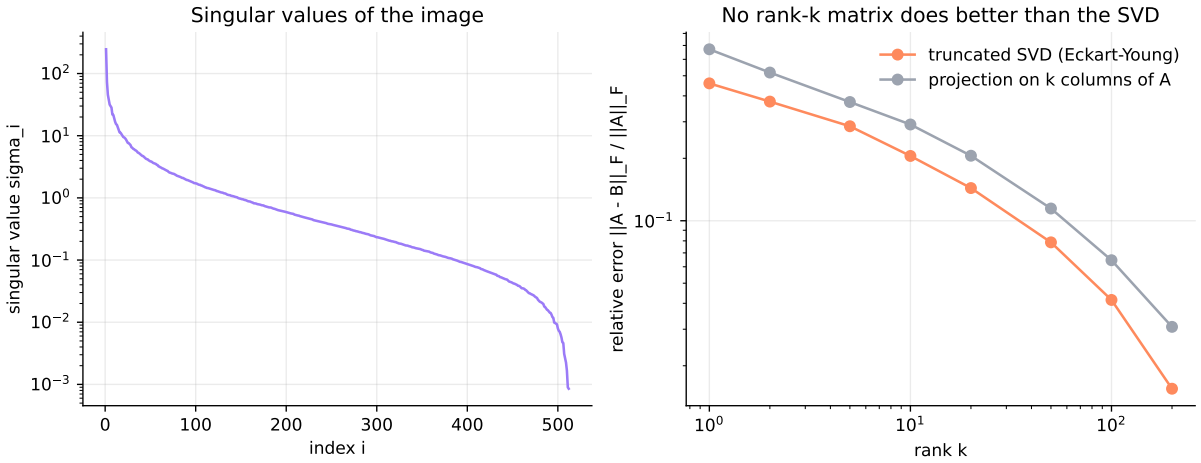


Figure 3 – À gauche, la décroissance des valeurs singulières : quelques-unes concentrent l'essentiel de l'énergie de l'image, ce qui rend la compression efficace. À droite, l'optimalité de la SVD : une autre approximation de rang k (projection sur k colonnes de l'image) fait toujours moins bien.

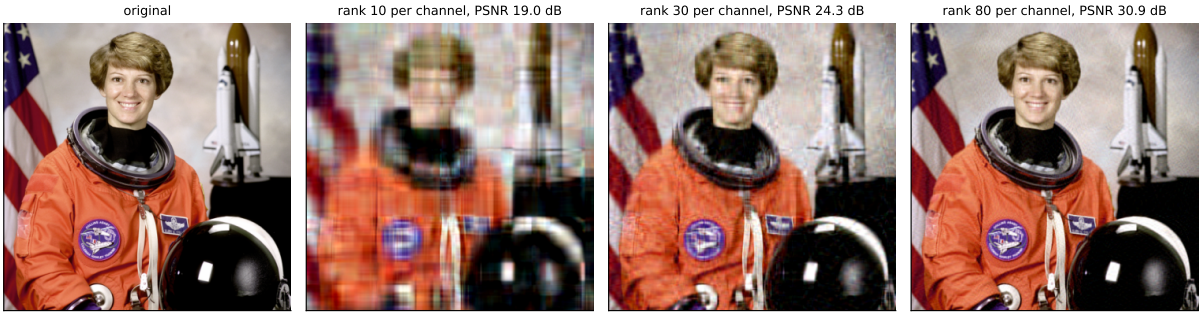


Figure 4 – Pour une image en couleurs, on applique la même méthode à chacun des trois canaux rouge, vert et bleu.

4 Conclusion

La même décomposition, la SVD, sert à deux usages : réduire la dimension d'un jeu de données en conservant le plus de variance possible (ACP), et approcher une matrice par une matrice de rang faible avec une erreur minimale et connue à l'avance (Eckart–Young). Les expériences confirment la théorie avec une précision proche de la précision machine, et montrent qu'il vaut mieux calculer la SVD de X que diagonaliser $X^\top X$. Prolongements possibles : les méthodes de SVD tronquée par itération (puissance itérée, Lanczos) ou randomisées, adaptées aux très grandes matrices, et l'ACP à noyau pour les structures non linéaires.

A Reproduire les résultats

Toutes les valeurs numériques de ce document sont générées par le code :

```
pip install -r requirements.txt
python experiments/run_all.py     # figures, results.md et report/generated.tex
pytest                            # 16 tests
cd report && pdflatex rapport.tex
```